Week 6 — Advanced Bash: Owning the System

Overview

This week elevated Bash scripting from basic commands to **security-focused automation**. The skills developed here mirror what both attackers and defenders use daily — writing scripts for reconnaissance, system auditing, and automated hardening. The phrase "owning the system" refers to mastering it completely through scripting, not unauthorized access.

Topics Covered

- Advanced shell scripting (conditionals, loops, functions)
- Variables, arrays, and string manipulation
- Script automation for system administration tasks
- Security auditing via script
- User activity monitoring
- Parsing and processing log output programmatically

Assignment

Developed a collection of Bash scripts designed to perform security-relevant system tasks, including:

- A **system audit script** that checks for unauthorized users, suspicious processes, world-writable files, and SUID binaries
- A **log parser** that extracts and summarizes failed login attempts, flagging IPs with excessive failures
- A **user management automation script** that creates multiple accounts, assigns groups, and enforces password policies in bulk

- A **backup automation script** integrated with cron for scheduled archiving

Script Examples

Iterate over users and check for suspicious sudo privileges

```bash
#!/bin/bash
echo "=== Checking sudo access ==="
for user in $(cut -f1 -d: /etc/passwd); do
    if sudo -l -U "$user" 2>/dev/null | grep -q "(ALL)"; then
        echo "[!] $user has full sudo access"
    fi
done
```

Find world-writable files (a common misconfiguration)

```bash
#!/bin/bash
echo "=== World-writable files ==="
find / -type f -perm -o+w 2>/dev/null | grep -v "/proc"
```

Extract top attacking IPs from auth log

```bash
#!/bin/bash
echo "=== Top Failed Login Sources ==="
grep "Failed password" /var/log/auth.log \
    | awk '{print $11}' \
    | sort | uniq -c | sort -rn \
    | head -10
```

Key Scripting Concepts

Concept	Description
`if/elif/else`	Conditional logic for decision-making
`for` / `while` loops	Iterating over files, users, or output
Functions	Reusable code blocks
`awk`	Field-based text parsing
`grep` + regex	Pattern matching in output and files
`sort` / `uniq`	Deduplication and frequency analysis
Exit codes	Checking if commands succeeded (`$?`)

Security Relevance

Scripting is a force multiplier for security work:

- **Blue team:** Automate log review, generate daily audit reports, bulk-harden configurations
- **Red team:** Automate reconnaissance steps, chain exploit payloads
- **SOC:** Parse thousands of log lines in seconds that would take hours manually